অধ্যায় ২২

২২

JavaScript Testing Mastery: Unit Testing, Integration Testing, Mocking, TDD এবং E2E

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- কোন আচরণ কোন স্তরের Test-এ (Unit, Integration, End-to-end) যাচাই করা উচিত, তা যুক্তি দিয়ে ঠিক করতে।
- Arrange–Act–Assert কাঠামো ও Table-driven Test দিয়ে পরিষ্কার, পড়তে সহজ Test লিখতে।
- Equivalence Partitioning ও Boundary Value Analysis দিয়ে কম Test-এ বেশি ত্রুটি ধরতে।
- Property-based Test দিয়ে হাজারো ইনপুটে একটি সাধারণ নিয়ম যাচাই করতে।
- Red–Green–Refactor চক্রে Test-Driven Development অনুশীলন করতে।
- `node:test`-এর `mock.fn`, `mock.method` ও `mock.timers` দিয়ে Network, সময় ও অপেক্ষা নিয়ন্ত্রণ করতে।
- স্থানীয় HTTP Server-এর বিরুদ্ধে Integration Test এবং আসল Browser-এ End-to-end Test লিখতে।
- Coverage Report পড়তে, এবং এর সীমা বুঝতে।
- একটি CI Pipeline-এ Test স্বয়ংক্রিয় করতে।
- একটি সম্পূর্ণ পরীক্ষিত **CurrencyDesk** Quote Service তৈরি করতে।

পূর্বশর্ত

- অধ্যায় ১৬: `node:test`, `assert`, Test Double-এর ধারণা, Testing Pyramid।
- অধ্যায় ১৯: Seed-যুক্ত এলোমেলো সংখ্যা।
- অধ্যায় ২০: স্থানীয় HTTP Server-এর বিরুদ্ধে Test।
- অধ্যায় ২১: Dependency Injection, Contract Test।

এই অধ্যায় কেন গুরুত্বপূর্ণ

একটি ভ্রমণ সংস্থার মুদ্রা-বিনিময় Counter-এর কথা ভাবুন। Software একজন গ্রাহককে বলে: ১,০০০ ডলার পাঠালে Fee কত, মোট কত দেবেন, এবং কত টাকা পাবেন। এখানে একটি পয়সার ভুলও গ্রাহকের অভিযোগ, হিসাবের গরমিল, বা নিয়ন্ত্রক সংস্থার প্রশ্ন ডেকে আনে। আর ভুলগুলো সাধারণত মাঝখানে লুকায় না; লুকায় **সীমায়** (৯৯৯.৯৯ বনাম ১,০০০.০০), **ব্যর্থতায়** (Rate Server বন্ধ), এবং **সময়ে** (Quote-এর মেয়াদ)।

অধ্যায় ১৬-এ আপনি Test লিখতে শিখেছেন। এই অধ্যায়ের প্রশ্ন আরেক ধাপ ওপরে: **কোন Test লিখবেন, কতগুলো, এবং কীভাবে নিশ্চিত হবেন যে সেগুলো আসলেই ত্রুটি ধরে?** উত্তরটি শুধু Tool নয়, একটি চিন্তার পদ্ধতি: ইনপুটকে শ্রেণিতে ভাগ করা, সীমা খোঁজা, নিয়ম হিসেবে আচরণ প্রকাশ করা, এবং বাইরের পৃথিবীকে (Network, ঘড়ি, Browser) নিয়ন্ত্রণে আনা।

২২.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
`node:test`, `assert`, Subtest (অধ্যায় ১৬)	Table-driven Test, `tap` Output পড়া, `t.after` দিয়ে পরিষ্কার
Stub, Spy, Mock, Fake-এর ধারণা (অধ্যায় ১৬)	`mock.fn`, `mock.method`, `mock.timers`
Coverage-এর পরিচয় (অধ্যায় ১৬)	Coverage Report থেকে বাস্তব ফাঁক খুঁজে Test যোগ
Seed-যুক্ত এলোমেলো ইনপুট (অধ্যায় ১৯)	Property-based Testing
Contract Test (অধ্যায় ২১)	Equivalence Partitioning, Boundary Value Analysis
—	TDD-এর পূর্ণ চক্র, আসল Browser-এ E2E, CI Pipeline

২২.২ কোন স্তরে কী যাচাই করবেন

Test-এর স্তর আলাদা প্রশ্নের উত্তর দেয়। একই আচরণ সব স্তরে যাচাই করা অপচয়; ভুল স্তরে যাচাই করা ঝুঁকি।

স্তর	প্রশ্ন	গতি ও স্থিরতা	CurrencyDesk-এ উদাহরণ
Unit	একটি Function কি নিজের নিয়ম মানে?	মিলিসেকেন্ড, সবচেয়ে স্থির	`feeFor`, `parseAmount`
Integration	অংশগুলো কি একসঙ্গে ঠিকমতো কথা বলে?	সেকেন্ডের ভগ্নাংশ	API থেকে Rate Server পর্যন্ত HTTP
End-to-end (E2E)	ব্যবহারকারী কি আসল Browser-এ কাজটি করতে পারেন?	সেকেন্ড, সবচেয়ে ভঙ্গুর	Form পূরণ, Click, ফল দেখা

সাধারণ নির্দেশিকা: **হিসাবের প্রতিটি নিয়ম Unit স্তরে** (দ্রুত, অনেক ইনপুট), **প্রতিটি সংযোগ ও Error-এর পথ Integration স্তরে** (সংখ্যায় কম), এবং **শুধু সবচেয়ে গুরুত্বপূর্ণ ব্যবহারকারীর পথ E2E স্তরে** (দুই-একটি)। E2E দিয়ে Fee-র ২০টি সীমা যাচাই করলে Test ধীর ও ভঙ্গুর হয়; Unit Test দিয়ে "Browser-এ Button কাজ করে" প্রমাণ করা যায় না।

২২.৩ Arrange–Act–Assert এবং Table-driven Test

প্রতিটি Test তিন ধাপে পড়া সহজ: **Arrange** (প্রস্তুতি), **Act** (একটি কাজ), **Assert** (প্রত্যাশা)। একই আচরণের অনেক ইনপুট থাকলে ইনপুট ও প্রত্যাশাকে একটি টেবিলে রাখুন এবং Loop-এ Test তৈরি করুন: প্রতিটি সারি আলাদা নামে রিপোর্ট হয়, তাই কোনটি ব্যর্থ তা সঙ্গে সঙ্গে দেখা যায়।

aaa.test.mjs · JavaScript — চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';

function applyDiscount(priceMinor, percent) {
  if (!Number.isInteger(percent) || percent < 0 || percent > 50) {
    throw new RangeError('ছাড় ০ থেকে ৫০%');
  }
  return priceMinor - Math.round((priceMinor * percent) / 100);
}

const rows = [
  [10_000, 0, 10_000],
  [10_000, 10, 9_000],
  [999, 50, 499],
];

for (const [price, percent, expected] of rows) {
  test(`${price} পয়সায় ${percent}% ছাড়`, () => {
    // Arrange: টেবিলের সারি। Act:
    const result = applyDiscount(price, percent);
    // Assert:
    assert.equal(result, expected);
  });
}

test('৫০%-এর বেশি ছাড় প্রত্যাখ্যাত', () => {
  assert.throws(() => applyDiscount(10_000, 51), RangeError);
});
```

একটি Test File সরাসরি `node` দিয়ে চালালে `node:test` TAP Format-এ ফল ছাপে। নিচে তার মূল লাইনগুলো (প্রতিটি Test-এর ফল ও সারাংশ); সময়ের (`duration_ms`) লাইনগুলো প্রতিবার বদলায় বলে বাদ দেওয়া:

Output

```
ok 1 - 10000 পয়সায় 0% ছাড়
ok 2 - 10000 পয়সায় 10% ছাড়
ok 3 - 999 পয়সায় 50% ছাড়
ok 4 - ৫০%-এর বেশি ছাড় প্রত্যাখ্যাত
# tests 4
# pass 4
# fail 0
```

তৃতীয় সারিটি লক্ষ করুন: ৯৯৯-এর ৫০% = ৪৯৯.৫, `Math.round` করে ৫০০ ছাড়, ফল ৪৯৯। এমন একটি সারিই প্রমাণ করে যে Rounding-এর নিয়ম ঠিক আছে—টেবিলে এই ধরনের "অস্বস্তিকর" ইনপুট রাখুন।

২২.৪ Equivalence Partitioning এবং Boundary Value Analysis

সব ইনপুট পরীক্ষা করা অসম্ভব। তাই ইনপুটকে **সমতুল্য শ্রেণিতে** (Equivalence Partition) ভাগ করুন—যে ইনপুটগুলোর সঙ্গে Code একই আচরণ করার কথা—এবং প্রতিটি শ্রেণি থেকে কয়েকটি প্রতিনিধি নিন। `parseAmount`-এর শ্রেণি: বৈধ পূর্ণসংখ্যা, দশমিকসহ, কমাসহ, সামনে-পেছনে ফাঁকা; এবং অবৈধ: ফাঁকা, অক্ষর, ঋণাত্মক, তিন দশমিক, ভুল কমা, বৈজ্ঞানিক রূপ।

তারপর **সীমা** খুঁজুন। ত্রুটি সবচেয়ে বেশি থাকে শ্রেণির সীমায়: `<` বনাম `<=`, শুরু বনাম শেষ। **Boundary Value Analysis** প্রতিটি সীমার ঠিক নিচে, ঠিক সীমায় এবং ঠিক ওপরে Test নেয়। CurrencyDesk-এর Fee-র তিনটি স্তর (১,০০০ ও ১০,০০০ টাকায় সীমা) দেখুন। Project-এর প্রথম খসড়ায় দ্বিতীয় স্তরের ন্যূনতম Fee ছিল ১৫ টাকা:

fee-cliff.js · JavaScript — চলবে: Browser Console ও Node.js

```
// খসড়া নিয়ম: ১,০০০ টাকার নিচে ২% (ন্যূনতম ৫০), তারপর ১.৫% (ন্যূনতম ১৫)।
function draftFee(minor) {
  if (minor < 100_000) {
    return Math.max(5_000, Math.round(minor * 0.02));
  }
  return Math.max(1_500, Math.round(minor * 0.015));
}

for (const minor of [99_999, 100_000, 100_001]) {
  const taka = (n) => (n / 100).toFixed(2);
  console.log(`${taka(minor)} পাঠালে Fee ${taka(draftFee(minor))}`);
}
```

Output

```
999.99 পাঠালে Fee 50.00
1000.00 পাঠালে Fee 15.00
1000.01 পাঠালে Fee 15.00
```

সীমার তিনটি Test একটি **ব্যবসার প্রশ্ন** তুলেছে, যা Code-এর ভুল নয়, নিয়মের ভুল: এক পয়সা বেশি পাঠালে গ্রাহক ৩৫ টাকা কম Fee দেন, তাই চতুর গ্রাহক ৯৯৯.৯৯-র বদলে ১,০০০ পাঠাবেন। এমন প্রশ্ন Test লেখার সময়ই ওঠে, এবং তার উত্তর দেয় ব্যবসা, Developer নয়। চূড়ান্ত Project-এ নিয়মটি বদলানো হয়েছে (অনুচ্ছেদ ২২.১৩)।

২২.৫ Property-based Testing

Example-based Test বলে "এই ইনপুটে এই ফল"। **Property-based Test** বলে "যেকোনো ইনপুটে এই নিয়ম সত্য", এবং অনেক এলোমেলো ইনপুটে তা যাচাই করে। Fee-র একটি স্বাভাবিক নিয়ম: **বেশি পাঠালে Fee কখনো কমে না**। খসড়া নিয়মের বিরুদ্ধে এই Property চালালে:

monotonic.test.mjs · JavaScript — চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';

function draftFee(minor) {
  if (minor < 100_000) {
    return Math.max(5_000, Math.round(minor * 0.02));
  }
  return Math.max(1_500, Math.round(minor * 0.015));
}

test('বেশি পাঠালে Fee কখনো কমে না', () => {
  let previous = 0;
  for (let minor = 1; minor <= 5_000_000; minor += 997) {
    const fee = draftFee(minor);
    assert.ok(fee >= previous, `${minor} পয়সায় Fee ${fee}, আগে ছিল ${previous}`);
    previous = fee;
  }
});
```

Error Message

```
not ok 1 - বেশি পাঠালে Fee কখনো কমে না
100698 পয়সায় Fee 1510, আগে ছিল 5000
# fail 1
```

Property-টি নিজেই সীমার কাছে প্রথম লঙ্ঘন খুঁজে বের করেছে—আমরা কোনো সীমা বলে দিইনি। এর শক্তি এখানেই: Property অজানা সীমাও ধরে। দুটি সতর্কতা: এলোমেলো ইনপুট ব্যবহার করলে **Seed** স্থির রাখুন (অধ্যায় ১৯), যাতে ব্যর্থ হলে একই ইনপুট আবার পাওয়া যায়; এবং Property সাধারণ ও সত্যিকারের নিয়ম হতে হবে, Implementation-এর হুবহু অনুলিপি নয়। CurrencyDesk-এর তিনটি Property: Round-trip (`format` তারপর `parse` আগের মান দেয়), Fee কখনো কমে না, এবং Fee সব সময় একটি নির্দিষ্ট পরিসরে। পূর্ণাঙ্গ Library (যেমন fast-check) ব্যর্থ ইনপুটকে ছোট করে (Shrinking) সবচেয়ে সরল উদাহরণ দেখায়; এখানে আমরা সেই Library ছাড়াই ধারণাটি দেখছি।

২২.৬ Test-Driven Development: Red, Green, Refactor

TDD-তে Test আগে লেখা হয়: **Red** (একটি ব্যর্থ Test, যা চাওয়া আচরণ বর্ণনা করে), **Green** (Test পাস করানোর সবচেয়ে সরল Code), **Refactor** (Test সবুজ রেখে Code পরিষ্কার)। এর মূল লাভ Test নয়, নকশা: আগে ব্যবহারকারীর দিক থেকে Function-কে ডাকতে হয় বলে Interface সরল হয়।

`formatAmount`-এর জন্য প্রথমে Test, এবং একটি ফাঁকা Function:

format.test.mjs · JavaScript — চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { formatAmount } from './format.mjs';

test('পয়সা থেকে হাজারের কমা ও দুই দশমিক', () => {
  assert.equal(formatAmount(123456, 'BDT'), '1,234.56 BDT');
  assert.equal(formatAmount(5, 'USD'), '0.05 USD');
});
```

format.mjs · JavaScript — চলবে: Node.js

```
export function formatAmount() {
  return '';
}
```

Terminal

```
node format.test.mjs
```

চালালে (Red):

red.mjs · JavaScript — চলবে: Node.js

```
await import('./format.test.mjs');
```

Error Message

```
not ok 1 - পয়সা থেকে হাজারের কমা ও দুই দশমিক
expected: '1,234.56 BDT'
actual: ''
```

ব্যর্থ হওয়াটা জরুরি: যে Test কখনো ব্যর্থ হয়নি, সে যে আসলে কিছু যাচাই করে, তার প্রমাণ নেই। এবার Green—সবচেয়ে সরল সঠিক Code:

format.mjs · JavaScript চলবে: Node.js

```
export function formatAmount(minor, currency) {
  const whole = Math.floor(minor / 100).toLocaleString('en-US');
  const cents = String(minor % 100).padStart(2, '0');
  return `${whole}.${cents} ${currency}`;
}
```

format.test.mjs · JavaScript চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { formatAmount } from './format.mjs';

test('পয়সা থেকে হাজারের কমা ও দুই দশমিক', () => {
  assert.equal(formatAmount(123456, 'BDT'), '1,234.56 BDT');
  assert.equal(formatAmount(5, 'USD'), '0.05 USD');
});
```

Output

```
ok 1 - পয়সা থেকে হাজারের কমা ও দুই দশমিক
# tests 1
# pass 1
# fail 0
```

পরের চক্রে একটি নতুন Test: ঋণাত্মক পরিমাণ (ফেরত টাকা)। এই Green সংস্করণে `-250` হতো `-3.-50` —আবার Red! Project-এর চূড়ান্ত `formatAmount` (অনুচ্ছেদ ২২.১৩) চিহ্ন আলাদা করে সেটি সামলায়; সেটিই Refactor-এর পর রূপ। প্রতিটি চক্র ছোট: একটি নতুন আচরণ, একটি নতুন Test।

২২.৭ node:test-এর Mock: Function, Method ও সময়

অধ্যায় ১৬-এ আপনি হাতে Test Double লিখেছেন। `node:test` নিজেই তিনটি হাতিয়ার দেয়:

- **`mock.fn(impl)`** : একটি Function, যা প্রতিটি ডাক মনে রাখে (`mock.calls` , `mock.callCount()`)।
- **`mock.method(object, 'name', impl)`** : একটি বিদ্যমান Object-এর Method সাময়িকভাবে বদলায়; Test শেষে `mock.restoreAll()` বা `t.mock` ব্যবহারে নিজে ফিরে আসে।
- **`mock.timers`** : `setTimeout` , `Date` ইত্যাদিকে নকল ঘড়িতে চালায়; `tick(ms)` দিয়ে সময় এগোয়, আসল অপেক্ষা ছাড়াই।

mocks.test.mjs · JavaScript চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
```

```
const notifier = { send: (to, text) => `SMS ${to}: ${text}` };

function remindLater(phone, delayMs) {
  const text = 'আপনার Quote-এর মেয়াদ শেষ হচ্ছে';
  setTimeout(() => notifier.send(phone, text), delayMs);
}

test('৬০ সেকেন্ড পরে ঠিক একটি SMS', (t) => {
  t.mock.timers.enable({ apis: ['setTimeout'] });
  const send = t.mock.method(notifier, 'send', () => 'sent');

  remindLater('01712345678', 60_000);
  t.mock.timers.tick(59_999);
  assert.equal(send.mock.callCount(), 0);

  t.mock.timers.tick(1);
  assert.equal(send.mock.callCount(), 1);
  assert.deepEqual(send.mock.calls[0].arguments,
    ['01712345678', 'আপনার Quote-এর মেয়াদ শেষ হচ্ছে']);
});

test('Test শেষে আসল Method ফিরে আসে', () => {
  assert.equal(notifier.send('x', 'y'), 'SMS x: y');
});
```

Output

```
ok 1 - ৬০ সেকেন্ড পরে ঠিক একটি SMS
ok 2 - Test শেষে আসল Method ফিরে আসে
# tests 2
# pass 2
# fail 0
```

এক মিনিটের অপেক্ষা Test-এ কয়েক মিলিসেকেন্ডে শেষ, এবং ৫৯,৯৯৯ মিলিসেকেন্ডে কিছু না ঘটা—একটি সীমার Test—আসল ঘড়িতে প্রায় অসম্ভব হতো। দ্বিতীয় Test প্রমাণ করে `t.mock` পরিষ্কার করেছে: একটি Test-এর Mock অন্য Test-এ চুঁইয়ে পড়েনি।

কতটা Mock? যা ধীর, অনিশ্চিত বা আপনার নিয়ন্ত্রণের বাইরে (Network, ঘড়ি, এলোমেলো সংখ্যা, ইমেইল), তা নকল করুন। আপনার নিজের বিশুদ্ধ Function নকল করবেন না: তখন Test শুধু প্রমাণ করে যে Mock নিজের মতো আচরণ করে। Mock যত বেশি, Test তত Implementation-এর সঙ্গে বাঁধা; তাই নির্ভরতা Inject করুন (অধ্যায় ২১) এবং সীমানায় নকল করুন।

২২.৮ Integration Test: আসল HTTP, নিয়ন্ত্রিত পৃথিবী

Unit Test-এ নকল `fetch` প্রমাণ করে যে Rate Client নিজের নিয়ম মানে, কিন্তু প্রমাণ করে না যে API আসলে HTTP-তে সঠিক Status ও JSON পাঠায়। Integration Test-এর জন্য CurrencyDesk একটি **স্থানীয় নকল Rate Provider** তৈরি করে—একটি আসল HTTP Server, যার আচরণ Test নিয়ন্ত্রণ করে (`failNext` দিয়ে ইচ্ছামতো 503)। নিয়মগুলো:

- **Port 0:** Operating System একটি খালি Port দেয়; একাধিক Test বা Developer-এর মধ্যে সংঘর্ষ নেই।
- **শুধু `127.0.0.1`:** Test-এর Server বাইরে থেকে পৌঁছানো যায় না।
- **`t.after(() => desk.close())`:** Test ব্যর্থ হলেও Server বন্ধ হয়; নইলে Process ঝুলে থাকে।
- **কোনো বাইরের API নয়:** আসল বিনিময়-হারের Service-এর ওপর নির্ভর করলে Test তার Uptime, Rate Limit ও প্রতিদিন বদলানো মানের ওপর নির্ভর করবে।

পূর্ণ Test অনুচ্ছেদ ২২.১২-এ।

২২.৯ Async Test, Cleanup এবং Flaky Test

যে Test কখনো পাস কখনো ব্যর্থ, তাকে **Flaky** বলে। Flaky Test-এর চেয়ে খারাপ কিছু কম আছে: দল ব্যর্থতাকে উপেক্ষা করতে শেখে, এবং একদিন আসল ত্রুটিও উপেক্ষিত হয়।

কারণ	উদাহরণ	সমাধান
আসল সময়	"১০০ ms অপেক্ষা করে দেখি"	`mock.timers`, অথবা Promise-এর অপেক্ষা
ভাগ করা অবস্থা	এক Test-এর Cache অন্য Test-এ	প্রতিটি Test নিজের Object বা Server তৈরি করে
Test-এর ক্রম	আগের Test-এর তৈরি তথ্যের ওপর নির্ভরতা	প্রতিটি Test স্বাধীন
বাইরের Network	আসল API	স্থানীয় নকল Server
`await` বাদ পড়া	Promise-এর Assertion Test শেষের পরে	`await assert.rejects(...)`
এলোমেলো ইনপুট	Seed ছাড়া `Math.random`	স্থির Seed
স্থির Port	দুটি Test একই Port চায়	Port 0

একটি সূক্ষ্ম ভুল: `assert.rejects(promise)` ফেরত দেয় একটি Promise; `await` না করলে Test আগেই "পাস" করে, এবং আসল ব্যর্থতা হারিয়ে যায়। নিয়ম: Async Test-এ প্রতিটি Promise-এর Assertion `await` করুন।

২২.১০ Coverage: কী বলে, কী বলে না

`node --test --experimental-test-coverage` প্রতিটি File-এর কত শতাংশ লাইন, শাখা (Branch) ও Function চলেছে তা দেখায়। CurrencyDesk-এ প্রথম চালানোর আসল Report-এর অংশ (এই বইয়ের লেখার সময়, Node.js 22.16.0):

Terminal

```
# file                 | line % | branch % | funcs % | uncovered lines
#  fees.mjs            | 100.00 |   100.00 |  100.00 |
#  money.mjs           |  94.59 |    92.31 |  100.00 | 21-22
#  quote.mjs           | 100.00 |   100.00 |   66.67 |
#  rates.mjs           | 100.00 |    96.67 |  100.00 |
```

Report দুটি আসল ফাঁক দেখিয়েছিল। `money.mjs`-এর ২১-২২ লাইন ("পরিমাণ খুব বড়") কোনো Test-এ চলেনি। আর `quote.mjs`-এ তিনটি Function-এর একটি— `isValid`, যা Quote-এর মেয়াদ যাচাই করে—একবারও ডাকা হয়নি: একটি ব্যবসার গুরুত্বপূর্ণ নিয়মের কোনো Test ছিল না। দুটির জন্য `tests/quote.test.mjs` যোগ করার পর এই File দুটি ১০০%।

কিন্তু উল্টোটি সত্য নয়: ১০০% Coverage মানে ত্রুটি নেই, তা নয়। খসড়া Fee-র নিয়মে প্রতিটি লাইন চলত, তবু ৩৫ টাকার "লাফ" ধরা পড়েছিল শুধু সীমার Test ও Property-তে। Coverage বলে **কোন Code-কে কোনো Test ছোঁয়নি**; বলে না Test ভালো কি না। তাই Coverage-কে লক্ষ্য না বানিয়ে ফাঁক খোঁজার যন্ত্র হিসেবে ব্যবহার করুন।

২২.১১ Mutation Testing: Test কি আসলেই ত্রুটি ধরে?

Coverage বলে কোন Code চলেছে; প্রশ্ন থেকে যায়, Code ভুল হলে Test কি ব্যর্থ হতো? **Mutation Testing** ঠিক এটি মাপে: Code-এ ছোট, ইচ্ছাকৃত ভুল (Mutant) ঢুকিয়ে Test চালানো হয়। কোনো Test ব্যর্থ হলে Mutant "ধরা পড়েছে" (Killed); সব Test পাস করলে Mutant "টিকে গেছে" (Survived)—অর্থাৎ সেই ধরনের ভুল আপনার Test-এর চোখ এড়িয়ে যাবে। নিচে Fee-র Function-এর চারটি Mutant, এবং Project-এর Boundary Test-এর আটটি সারি:

mutation.js · JavaScript — চলবে: Browser Console ও Node.js

```
const TIERS = [
  { from: 1, bp: 200, min: 5_000 },
  { from: 100_000, bp: 150, min: 5_000 },
  { from: 1_000_000, bp: 100, min: 15_000 },
];
const MAX = 50_000_000;

// mutant: Code-এ একটি ছোট, ইচ্ছাকৃত পরিবর্তন।
function makeFee(mutant = '') {
  return (minor) => {
    const over = mutant === 'max>=' ? minor >= MAX : minor > MAX;
    if (minor < 1 || over) {
      throw new RangeError('সীমার বাইরে');
    }
    const fits = (t) => (mutant === 'from>' ? minor > t.from : minor >= t.from);
    const tier = TIERS.findLast(fits) ?? TIERS[0];
    const min = mutant === 'cliff' && tier.from === 100_000 ? 1_500 : tier.min;
    const half = mutant === 'truncate' ? 0n : 5_000n;
    const percent = Number((BigInt(minor) * BigInt(tier.bp) + half) / 10_000n);
```

```
    return Math.max(min, percent);
  };
}

const rows = [[1, 5_000], [99_999, 5_000], [100_000, 5_000],
  [333_334, 5_000], [333_367, 5_001], [999_999, 15_000],
  [1_000_000, 15_000], [MAX, 500_000]];

const killedBy = (fee) => rows.find(([minor, expected]) => {
  try {
    return fee(minor) !== expected;
  } catch {
    return true;
  }
});

console.log(`আসল Code: ${killedBy(makeFee()) ? 'ব্যর্থ' : 'সব পাস'}`);
for (const mutant of ['cliff', 'truncate', 'max>=', 'from>']) {
  const row = killedBy(makeFee(mutant));
  console.log(`${mutant.padEnd(8)} ${row ? `ধরা পড়েছে (${row[0]} পয়সায়)` : 'টিকে
      গেছে'}`);
}
```

Output

```
আসল Code: সব পাস
cliff    ধরা পড়েছে (100000 পয়সায়)
truncate ধরা পড়েছে (333367 পয়সায়)
max>=    ধরা পড়েছে (50000000 পয়সায়)
from>    টিকে গেছে
```

তিনটি Mutant ধরা পড়েছে, এবং প্রতিটি ঠিক সেই সারিতে, যেটি তার জন্যই লেখা: পুরোনো "লাফ" (`cliff`) ১,০০০ টাকায়, Rounding বাদ দেওয়া (`truncate`) ৩,৩৩৩.৬৭-এ, এবং সর্বোচ্চ সীমার `>` থেকে `>=` সর্বোচ্চ পরিমাণে। কিন্তু স্তর বাছাইয়ের `>=` থেকে `>` টিকে গেছে। কারণ খুঁজলে দেখা যায়, চূড়ান্ত Fee-র টেবিল সীমায় অবিচ্ছিন্ন: ১,০০০ টাকায় প্রথম ও দ্বিতীয় স্তর দুটিই ৫০ টাকা দেয়, ১০,০০০ টাকায় দ্বিতীয় ও তৃতীয় স্তর দুটিই ১৫০। তাই এই Mutant আচরণ বদলায়ই না—একে বলে **Equivalent Mutant**, এবং এটি কোনো Test-এর দুর্বলতা নয়। টিকে যাওয়া প্রতিটি Mutant এভাবে বিশ্লেষণ করতে হয়: হয় একটি অনুপস্থিত Test, নয়তো একটি সমতুল্য পরিবর্তন।

বাস্তব Project-এ এই কাজ হাতে করা হয় না; StrykerJS-এর মতো Tool শত শত Mutant স্বয়ংক্রিয়ভাবে তৈরি করে এবং একটি "Mutation Score" দেয়। এটি ধীর (প্রতিটি Mutant-এ পুরো Test Suite), তাই সাধারণত গুরুত্বপূর্ণ Module-এ বা রাতের CI-তে চালানো হয়। এখানে আমরা Tool ছাড়াই ধারণাটি দেখলাম: **Coverage বলে Test কোথায় গেছে, Mutation বলে Test কী দেখেছে।**

‘‘

২২.১২ End-to-end: আসল Browser-এ

E2E Test একজন ব্যবহারকারীর মতো আচরণ করে: Page খোলে, Form পূরণ করে, Button চাপে, পর্দায় ফল দেখে। এটি এমন ত্রুটি ধরে যা অন্য কোনো স্তর পারে না: ভুল `id`, Script লোড না হওয়া, JSON-এর Field-এর নাম Server ও Browser-এ অমিল।

পেশাদার দলে সাধারণত Playwright বা Cypress ব্যবহৃত হয়। এগুলো Install করতে হয়; এই বই কোনো Package Install না করে কাজ করে, তাই CurrencyDesk-এ একটি ছোট নিজস্ব Helper (`tests/support/browser.mjs`) আছে, যা Chrome-এর **DevTools Protocol** দিয়ে Headless Browser চালায়। Playwright ঠিক এই Protocol-ই ব্যবহার করে, অনেক বেশি সুবিধাসহ (স্বয়ংক্রিয় অপেক্ষা, একাধিক Browser, Trace, Screenshot)। বাস্তব Project-এ Playwright ব্যবহার করুন; এখানের Helper শুধু দেখায় ভেতরে কী ঘটে।

E2E Test-এর নিয়ম: আসল Server চালু করে, Browser শুধু সেটিতে যায়; কোনো বাইরের Site নয়; প্রতিটি Test নিজের তথ্য তৈরি করে; এবং `setTimeout`-এ নির্দিষ্ট সময় অপেক্ষার বদলে **একটি শর্তের জন্য অপেক্ষা** (Result-এর `data-state` বসা পর্যন্ত)—এটিই E2E Flakiness কমানোর সবচেয়ে বড় কৌশল। Chrome বা Edge না পাওয়া গেলে Test কারণসহ **Skip** হয়, ব্যর্থ নয়, যাতে Browser-হীন যন্ত্রেও বাকি Test চলে।

২২.১৩ Project: CurrencyDesk

CurrencyDesk একটি ভ্রমণ সংস্থার মুদ্রা-বিনিময় Counter-এর Quote Service: গ্রাহক কোন মুদ্রা থেকে কোন মুদ্রায় কত পাঠাতে চান দিলে, Service Fee, মোট দেয়, পাওয়া পরিমাণ এবং মেয়াদ জানায়। হার আসে একটি Rate Provider থেকে (এখানে স্থানীয় নকল Server)। Project-এর আসল বিষয়বস্তু তার Test: ৩৮টি, পাঁচ ধরনের।

> **নোট:** CurrencyDesk একটি শিক্ষামূলক Demo। হারগুলো কাল্পনিক নমুনা, Server শুধু `127.0.0.1`-এ চলে, এবং বাস্তব অর্থ-লেনদেনের জন্য প্রয়োজনীয় নিয়ন্ত্রণ (Authentication, Audit, আইনি অনুমোদন, আসল Provider-এর চুক্তি) এতে নেই।

File-এর কাঠামো

Folder-এর কাঠামো

```
currency-desk/
├── server.mjs              চালু করে (127.0.0.1)
├── package.json            npm test ও npm run coverage
├── .github/workflows/test.yml   CI
├── src/
│   ├── money.mjs           পয়সায় হিসাব, parse, format, convert
│   ├── fees.mjs            স্তরভিত্তিক Fee
│   ├── rates.mjs           Rate Client: Cache, Retry
│   ├── quote.mjs           Quote Service
│   ├── rate-server.mjs     স্থানীয় নকল Rate Provider
│   ├── api.mjs             HTTP API ও Static File
│   └── start.mjs           দুই Server জোড়া লাগানো
├── public/                 index.html, app.mjs
└── tests/                  ৭টি Test File, support/browser.mjs
```

src/money.mjs ও src/fees.mjs

money.mjs · JavaScript চলবে: Node.js

```javascript
// টাকা সব সময় পূর্ণসংখ্যায় (Minor Unit: পয়সা বা সেন্ট)। 0.1 + 0.2-এর মতো
// Floating-point ভুল এখানে ঢোকে না।
const AMOUNT = /^(?:\d{1,3}(?:,\d{3})+|\d+)(?:\.\d{1,2})?$/;

export class MoneyError extends Error {
  constructor(message) {
    super(message);
    this.name = 'MoneyError';
  }
}

// "1,234.5" থেকে 123450। কমা শুধু সঠিক তিন অঙ্কের দলে।
export function parseAmount(text) {
  const clean = String(text ?? '').trim();
  if (!AMOUNT.test(clean)) {
    throw new MoneyError(`অবৈধ পরিমাণ: "${clean}"`);
  }
  const [whole, fraction = ''] = clean.replaceAll(',', '').split('.');
  const minor = Number(whole) * 100 + Number(fraction.padEnd(2, '0'));
  if (!Number.isSafeInteger(minor)) {
    throw new MoneyError('পরিমাণ খুব বড়');
  }
  return minor;
}

export function formatAmount(minor, currency) {
  const sign = minor < 0 ? '-' : '';
```

```
  const abs = Math.abs(minor);
  const whole = Math.floor(abs / 100).toLocaleString('en-US');
  return `${sign}${whole}.${String(abs % 100).padStart(2, '0')} ${currency}`;
}

// হার Micro Unit-এ (1 = 0.000001)। গুণফল BigInt-এ, অর্ধেক হলে ওপরে।
export function convert(minor, rateMicro) {
  const scaled = BigInt(minor) * BigInt(rateMicro) + 500_000n;
  return Number(scaled / 1_000_000n);
}
```

fees.mjs · JavaScript চলবে: Node.js

```
// Fee-র স্তর। from ও MAX_AMOUNT পয়সায়; bp = Basis Point (100 bp = 1%)।
// প্রতিটি স্তরের নিচের সীমা অন্তর্ভুক্ত। min এমনভাবে বাছা যে বড় পরিমাণে
// Fee কখনো কমে না (স্তরের সীমায় "লাফ" নেই)।
export const FEE_TIERS = Object.freeze([
  Object.freeze({ from: 1, bp: 200, min: 5_000 }),
  Object.freeze({ from: 100_000, bp: 150, min: 5_000 }),
  Object.freeze({ from: 1_000_000, bp: 100, min: 15_000 }),
]);

export const MAX_AMOUNT = 50_000_000;

export function feeFor(minor) {
  if (!Number.isSafeInteger(minor) || minor < 1) {
    throw new RangeError('পরিমাণ অন্তত 0.01 হতে হবে');
  }
  if (minor > MAX_AMOUNT) {
    throw new RangeError('এক লেনদেনে সর্বোচ্চ 500,000.00');
  }
  const tier = FEE_TIERS.findLast((t) => minor >= t.from);
  const percent = Number((BigInt(minor) * BigInt(tier.bp) + 5_000n) / 10_000n);
  return Math.max(tier.min, percent);
}
```

টাকা পয়সায় পূর্ণসংখ্যা হিসেবে, হার Micro Unit-এ (১২১.৪৫ = ১২১,৪৫০,০০০), এবং গুণ `BigInt`-এ: ৯০ কোটি টাকার পয়সা × হার সাধারণ Number-এর নিরাপদ সীমা (`Number.MAX_SAFE_INTEGER`) ছাড়িয়ে যায়। চূড়ান্ত Fee-র টেবিলে দ্বিতীয় স্তরের ন্যূনতম ৫০ টাকা, তাই ১,০০০ টাকায় আর লাফ নেই; Property Test এখন পাস করে।

src/rates.mjs

rates.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
export class RateError extends Error {
  constructor(code, message, { retryable = false, cause } = {}) {
    super(message, { cause });
    this.name = 'RateError';
```

```
    this.code = code;
    this.retryable = retryable;
  }
}

const CURRENCY = /^[A-Z]{3}$/;
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

// সব বাইরের নির্ভরতা Inject করা: fetch, ঘড়ি, অপেক্ষা। Test-এ সবই বদলানো যায়।
export function createRateClient(options) {
  const { baseUrl, fetchFn = fetch, now = Date.now, sleep = wait } = options;
  const { ttlMs = 60_000, retries = 2 } = options;
  const cache = new Map();

  async function fetchOnce(pair) {
    let response;
    try {
      response = await fetchFn(`${baseUrl}/rates/${pair}`);
    } catch (error) {
      throw new RateError('NETWORK', 'Rate Server-এ পৌঁছানো যায়নি',
        { retryable: true, cause: error });
    }
    if (response.status === 404) {
      throw new RateError('UNKNOWN_PAIR', `${pair}-এর হার নেই`);
    }
    if (!response.ok) {
      throw new RateError('UPSTREAM', `Rate Server: ${response.status}`,
        { retryable: response.status >= 500 });
    }
    const { rateMicro } = await response.json();
    if (!Number.isSafeInteger(rateMicro) || rateMicro <= 0) {
      throw new RateError('BAD_DATA', 'Rate Server অবৈধ হার পাঠিয়েছে');
    }
    return rateMicro;
  }

// ...
```

rates.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

  async function getRate(from, to) {
    if (!CURRENCY.test(from) || !CURRENCY.test(to) || from === to) {
      throw new RateError('BAD_CURRENCY', 'মুদ্রা তিন অক্ষরের, এবং দুটি ভিন্ন');
    }
    const pair = `${from}-${to}`;
    const hit = cache.get(pair);
    if (hit && now() - hit.at < ttlMs) {
```

```
      return hit.rateMicro;
    }
    for (let attempt = 0; ; attempt += 1) {
      try {
        const rateMicro = await fetchOnce(pair);
        cache.set(pair, { rateMicro, at: now() });
        return rateMicro;
      } catch (error) {
        if (!error.retryable || attempt >= retries) {
          throw error;
        }
        await sleep(100 * 2 ** attempt);
      }
    }
  }

  return { getRate };
}
```

Testability-ই এই File-এর নকশা ঠিক করেছে: `fetchFn`, `now`, `sleep` তিনটিই Inject করা, Default আসল। Error-এর `retryable` চিহ্ন বলে কোনটি আবার চেষ্টাযোগ্য (Network, 5xx) আর কোনটি নয় (404, অবৈধ মুদ্রা, অবৈধ তথ্য)। Retry-র অপেক্ষা দ্বিগুণ হয় (১০০, ২০০ ms)। Cache-এর মেয়াদ এক মিনিট।

src/quote.mjs, src/rate-server.mjs ও src/api.mjs

quote.mjs · JavaScript চলবে: Node.js

```
import { parseAmount, convert } from './money.mjs';
import { feeFor } from './fees.mjs';

// একটি Quote: কত পাঠাবেন, Fee, মোট দেবেন, কত পাবেন, এবং কতক্ষণ বৈধ।
export function createQuoteService(deps) {
  const { rates, newId, now = Date.now, validityMs = 60_000 } = deps;

  async function quote({ from, to, amount }) {
    const minor = parseAmount(amount);
    const fee = feeFor(minor);
    const rateMicro = await rates.getRate(from, to);
    return {
      id: newId(),
      from,
      to,
      amount: minor,
      fee,
      payable: minor + fee,
      receives: convert(minor, rateMicro),
      rateMicro,
      expiresAt: now() + validityMs,
```

```
    };
  }

  const isValid = (q) => now() < q.expiresAt;
  return { quote, isValid };
}
```

rate-server.mjs · JavaScript চলবে: Node.js

```
import { createServer } from 'node:http';

// একটি স্থানীয় নকল Rate Provider (Test ও Demo-র জন্য; বাস্তব Provider নয়)।
// controls দিয়ে Test ইচ্ছামতো ব্যর্থতা তৈরি করতে পারে।
export function createRateServer(table) {
  const controls = { failNext: 0, failStatus: 503, requests: 0 };
  const server = createServer((req, res) => {
    controls.requests += 1;
    const pair = /^\/rates\/([A-Z]{3}-[A-Z]{3})$/.exec(req.url)?.[1];
    if (controls.failNext > 0) {
      controls.failNext -= 1;
      res.writeHead(controls.failStatus).end();
    } else if (pair && Object.hasOwn(table, pair)) {
      res.writeHead(200, { 'content-type': 'application/json' });
      res.end(JSON.stringify({ pair, rateMicro: table[pair] }));
    } else {
      res.writeHead(404).end();
    }
  });
  return { server, controls };
}
```

api.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
import { createServer } from 'node:http';
import { readFile } from 'node:fs/promises';
import { MoneyError } from './money.mjs';

const PUBLIC = new URL('../public/', import.meta.url);
const STATIC = { '/': 'index.html', '/app.mjs': 'app.mjs' };
const TYPES = { html: 'text/html', mjs: 'text/javascript' };
const STATUS = { BAD_CURRENCY: 400, UNKNOWN_PAIR: 404 };

function send(res, status, data) {
  res.writeHead(status, { 'content-type': 'application/json; charset=utf-8' });
  res.end(JSON.stringify(data));
}

// Error-এর ধরন থেকে HTTP Status: ব্যবহারকারীর ভুল 4xx, বাইরের সমস্যা 502।
```

```
function statusFor(error) {
  if (error instanceof MoneyError || error instanceof RangeError) {
    return 400;
  }
  return STATUS[error.code] ?? (error.name === 'RateError' ? 502 : 500);
}

// ...
```

api.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

export function createApi({ quotes }) {
  return createServer(async (req, res) => {
    try {
      if (req.method === 'GET' && Object.hasOwn(STATIC, req.url)) {
        const file = STATIC[req.url];
        const type = TYPES[file.split('.').pop()];
        res.writeHead(200, { 'content-type': `${type}; charset=utf-8` });
        res.end(await readFile(new URL(file, PUBLIC)));
      } else if (req.method === 'POST' && req.url === '/api/quotes') {
        let body = '';
        for await (const chunk of req) {
          body += chunk;
        }
        send(res, 201, await quotes.quote(JSON.parse(body || '{}')));
      } else {
        send(res, 404, { error: 'পাওয়া যায়নি' });
      }
    } catch (error) {
      const status = error instanceof SyntaxError ? 400 : statusFor(error);
      const message = status === 500 ? 'Server-এ সমস্যা' : error.message;
      send(res, status, { error: message, code: error.code ?? null });
    }
  });
}
```

`quote` প্রথমে পরিমাণ যাচাই করে, তারপর Rate Server-এ যায়: অবৈধ ইনপুটে বাইরের Call-এর খরচ নেই (একটি Test এটি প্রমাণ করে)। API-র `statusFor` Error-এর ধরন থেকে HTTP Status ঠিক করে: ব্যবহারকারীর ভুল 400, অজানা মুদ্রা-জোড়া 404, Provider-এর সমস্যা 502 (Bad Gateway), অপ্রত্যাশিত Error 500 সাধারণ বার্তায়।

src/start.mjs ও server.mjs

start.mjs · JavaScript চলবে: Node.js

```
import { createRateServer } from './rate-server.mjs';
import { createRateClient } from './rates.mjs';
```

```
import { createQuoteService } from './quote.mjs';
import { createApi } from './api.mjs';

export const SAMPLE_RATES = Object.freeze({
  'USD-BDT': 121_450_000,
  'BDT-USD': 8_234,
  'EUR-BDT': 131_900_000,
  'SAR-BDT': 32_380_000,
});

const listen = (server) => new Promise((resolve) => {
  server.listen(0, '127.0.0.1', () => resolve(server.address().port));
});

// দুটি স্থানীয় Server (নকল Rate Provider ও Quote API), এলোমেলো Port-এ।
export async function startDesk({ table = SAMPLE_RATES, retries = 2 } = {}) {
  const provider = createRateServer(table);
  const ratePort = await listen(provider.server);
  let counter = 0;
  const rates = createRateClient({
    baseUrl: `http://127.0.0.1:${ratePort}`, retries, sleep: async () => {},
  });
  const quotes = createQuoteService({
    rates, newId: () => `Q-${String(++counter).padStart(4, '0')}`,
  });
  const api = createApi({ quotes });
  const apiPort = await listen(api);
  const close = () => Promise.all([provider.server, api].map((s) =>
    new Promise((resolve) => s.close(resolve))));
  return { url: `http://127.0.0.1:${apiPort}`, provider, close };
}
```

server.mjs · JavaScript — চলবে: Node.js

```
import { startDesk } from './src/start.mjs';

// শুধু 127.0.0.1-এ; হারগুলো কাল্পনিক নমুনা, বাস্তব বিনিময় হার নয়।
const desk = await startDesk();
console.log(`CurrencyDesk (শিক্ষামূলক Demo): ${desk.url}`);
console.log('বন্ধ করতে Ctrl+C চাপুন।');

process.on('SIGINT', async () => {
  await desk.close();
  process.exit(0);
});
```

`startDesk` একটি Composition Root (অধ্যায় ২১), যা Demo এবং প্রতিটি Integration ও E2E Test একইভাবে ব্যবহার করে: তাই Test আসল জোড়া লাগানো পরীক্ষা করে, একটি আলাদা Test-সংস্করণ নয়। Test-এ Retry-র অপেক্ষা শূন্য (`sleep: async () => {}`), যাতে ব্যর্থতার পথ দ্রুত চলে।

Browser UI

index.html · HTML — চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>CurrencyDesk</title>
  <script type="module" src="/app.mjs"></script>
</head>
<body>
  <h1>CurrencyDesk <small>শিক্ষামূলক Demo</small></h1>
  <form id="quote-form">
    <label>থেকে <input id="from" value="USD" maxlength="3"></label>
    <label>প্রতি <input id="to" value="BDT" maxlength="3"></label>
    <label>পরিমাণ <input id="amount" inputmode="decimal"></label>
    <button id="get-quote">Quote দেখুন</button>
  </form>
  <p id="result" role="status"></p>
</body>
</html>
```

app.mjs · JavaScript — চলবে: Browser

```javascript
const $ = (selector) => document.querySelector(selector);
const money = (minor, currency) => {
  const options = { minimumFractionDigits: 2 };
  return `${(minor / 100).toLocaleString('en-US', options)} ${currency}`;
};

$('#quote-form').addEventListener('submit', async (event) => {
  event.preventDefault();
  const body = {
    from: $('#from').value.trim().toUpperCase(),
    to: $('#to').value.trim().toUpperCase(),
    amount: $('#amount').value,
  };
  const response = await fetch('/api/quotes', {
    method: 'POST',
    headers: { 'content-type': 'application/json' },
    body: JSON.stringify(body),
  });
```

```
  const data = await response.json();
  $('#result').dataset.state = response.ok ? 'ok' : 'error';
  $('#result').textContent = response.ok
    ? `দেবেন ${money(data.payable, data.from)}`
      + ` (Fee ${money(data.fee, data.from)}),`
      + ` পাবেন ${money(data.receives, data.to)}`
    : data.error;
});
```

ফল বসানোর সময় `data-state` (`ok` বা `error`) চিহ্ন: E2E Test এটির জন্য অপেক্ষা করে, অনুমান করা সময়ের জন্য নয়।

Test: সীমা, Mock ও Property

fees.test.mjs · JavaScript — চলবে: Node.js

```
// Boundary Value Analysis: প্রতিটি সীমার ঠিক নিচে, ঠিক ওপরে ও ঠিক সীমায়।
import test from 'node:test';
import assert from 'node:assert/strict';
import { feeFor, MAX_AMOUNT } from '../src/fees.mjs';

const cases = [
  ['সবচেয়ে ছোট বৈধ', 1, 5_000],
  ['প্রথম স্তরের শেষ (999.99)', 99_999, 5_000],
  ['দ্বিতীয় স্তরের শুরু (1,000.00)', 100_000, 5_000],
  ['1.5% যেখানে ন্যূনতমকে ছাড়ায় (3,333.34)', 333_334, 5_000],
  ['তার পরের পয়সা (3,333.67)', 333_367, 5_001],
  ['দ্বিতীয় স্তরের শেষ (9,999.99)', 999_999, 15_000],
  ['তৃতীয় স্তরের শুরু (10,000.00)', 1_000_000, 15_000],
  ['সর্বোচ্চ (500,000.00)', MAX_AMOUNT, 500_000],
];

for (const [name, minor, fee] of cases) {
  test(`Fee: ${name}`, () => {
    assert.equal(feeFor(minor), fee);
  });
}

test('সীমার বাইরে: 0, ঋণাত্মক, সর্বোচ্চের এক পয়সা বেশি, ভগ্নাংশ', () => {
  for (const minor of [0, -1, MAX_AMOUNT + 1, 1.5, Number.NaN]) {
    assert.throws(() => feeFor(minor), RangeError, String(minor));
  }
});
```

rates.test.mjs · JavaScript — নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) — চলবে: Node.js

```
// Test Double: mock.fn দিয়ে নকল fetch, নকল ঘড়ি, এবং mock.timers।
import test, { mock } from 'node:test';
```

```
import assert from 'node:assert/strict';
import { createRateClient } from '../src/rates.mjs';

const reply = (status, body = {}) =>
  new Response(JSON.stringify(body), { status });
const ok = (rateMicro) => reply(200, { rateMicro });

test('Cache: TTL-এর মধ্যে দ্বিতীয় অনুরোধ Network-এ যায় না', async () => {
  let time = 0;
  const fetchFn = mock.fn(async () => ok(121_450_000));
  const rates = createRateClient(
    { baseUrl: 'http://x', fetchFn, now: () => time });
  await rates.getRate('USD', 'BDT');
  time = 59_999;
  await rates.getRate('USD', 'BDT');
  assert.equal(fetchFn.mock.callCount(), 1);
  time = 60_000;
  await rates.getRate('USD', 'BDT');
  assert.equal(fetchFn.mock.callCount(), 2);
  assert.equal(fetchFn.mock.calls[0].arguments[0], 'http://x/rates/USD-BDT');
});

test('Retry: 503 দুবার, তৃতীয়বার সফল; অপেক্ষা 100 ও 200 ms', async () => {
  const responses = [reply(503), reply(503), ok(8_234)];
  const fetchFn = mock.fn(async () => responses.shift());
  const sleep = mock.fn(async () => {});
  const rates = createRateClient({ baseUrl: 'http://x', fetchFn, sleep });
  assert.equal(await rates.getRate('BDT', 'USD'), 8_234);
  assert.deepEqual(sleep.mock.calls.map((c) => c.arguments[0]), [100, 200]);
});

// ...
```

property.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// ...

test('Fee কখনো কমে না: বড় পরিমাণে Fee সমান বা বেশি', () => {
  let previous = 0;
  for (let minor = 1; minor <= MAX_AMOUNT; minor += 997) {
    const fee = feeFor(minor);
    assert.ok(fee >= previous, `${minor} পয়সায় Fee ${fee} < ${previous}`);
    previous = fee;
  }
});

// ...
```

`fees.test.mjs`-এর প্রতিটি সারি একটি সীমার নাম বলে, তাই ব্যর্থ হলে Report-ই বলে দেয় কোন সীমা ভেঙেছে। `rates.test.mjs`-এর Retry Test দুটি জিনিস প্রমাণ করে: তৃতীয় চেষ্টায় সফল, এবং অপেক্ষা ঠিক ১০০ ও ২০০ ms—কোনো আসল অপেক্ষা ছাড়াই।

Test: Integration ও E2E

integration.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// Integration Test: আসল HTTP, আসল দুই Server (শুধু 127.0.0.1, এলোমেলো Port)।
import test from 'node:test';
import assert from 'node:assert/strict';
import { startDesk } from '../src/start.mjs';

async function post(desk, body) {
  const response = await fetch(`${desk.url}/api/quotes`, {
    method: 'POST',
    headers: { 'content-type': 'application/json' },
    body: typeof body === 'string' ? body : JSON.stringify(body),
  });
  return { status: response.status, data: await response.json() };
}

test('Quote API', async (t) => {
  const desk = await startDesk();
  t.after(() => desk.close());

  await t.test('201: সম্পূর্ণ Quote, সব টাকা পয়সায়', async () => {
    const { status, data } = await post(desk,
      { from: 'USD', to: 'BDT', amount: '1,000' });
    assert.equal(status, 201);
    assert.equal(data.id, 'Q-0001');
    assert.equal(data.fee, 5_000);
    assert.equal(data.payable, 105_000);
    assert.equal(data.receives, 12_145_000);
    assert.ok(data.expiresAt > Date.now());
  });

// ...

  await t.test('502: Provider বারবার ব্যর্থ', async () => {
    desk.provider.controls.failNext = 3;
    const { status, data } = await post(desk,
      { from: 'EUR', to: 'BDT', amount: '5' });
    assert.equal(status, 502);
    assert.equal(data.code, 'UPSTREAM');
  });

// ...
```

e2e.test.mjs · JavaScript চলবে: Node.js

```
// End-to-end: আসল Browser, আসল Server, ব্যবহারকারীর মতো Form পূরণ ও Click।
// Chrome বা Edge না থাকলে Skip (কারণসহ), ব্যর্থ নয়।
import test from 'node:test';
import assert from 'node:assert/strict';
import { startDesk } from '../src/start.mjs';
import { findBrowser, launch } from './support/browser.mjs';

const skip = findBrowser()
  ? false
  : 'Chrome বা Edge পাওয়া যায়নি (CHROME_PATH দিন)';

test('Browser-এ Quote', { skip, timeout: 60_000 }, async (t) => {
  const desk = await startDesk();
  const page = await launch();
  t.after(async () => {
    await page.close();
    await desk.close();
  });
  await page.open(desk.url);
  const waitForResult = () => page.eval(`new Promise((resolve) => {
    const tick = () => document.querySelector('#result').dataset.state
      ? resolve(document.querySelector('#result').textContent)
      : setTimeout(tick, 50);
    tick();
  })`);

  await page.type('#amount', '1,000');
  await page.click('#get-quote');
  assert.equal(await waitForResult(),
    'দেবেন 1,050.00 USD (Fee 50.00 USD), পাবেন 121,450.00 BDT');

  await page.eval("delete document.querySelector('#result').dataset.state");
  await page.type('#amount', '1.234');
  await page.click('#get-quote');
  assert.equal(await waitForResult(), 'অবৈধ পরিমাণ: "1.234"');
});
```

Integration Test একটি Parent Test-এর ভেতরে Subtest, যাতে দুই Server একবার চালু হয় এবং `t.after`-এ একবার বন্ধ হয়। `failNext = 2` দুটি 503-এর পর সফলতা দেয় (Retry পাস), `failNext = 3` সব চেষ্টা ব্যর্থ করে (502)। E2E Test আসল Browser-এ Form পূরণ করে দুটি পথ যাচাই করে: সফল Quote, এবং অবৈধ পরিমাণের বার্তা।

CI

test.yml · YAML — চলবে: Node.js

```yaml
# প্রতিটি Push ও Pull Request-এ Test (GitHub Actions)। এই বইয়ের QA-তে
# এই File-টি GitHub-এ চালানো হয়নি; একই Command স্থানীয়ভাবে চালানো হয়েছে।
name: CurrencyDesk tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '22'
      - run: node --test
      - run: node --test --experimental-test-coverage
```

এই Workflow GitHub-এ প্রতিটি Push ও Pull Request-এ Test চালায়, যাতে ত্রুটিপূর্ণ Code Merge হওয়ার আগেই ধরা পড়ে। এই বইয়ের QA-তে File-টি GitHub-এ চালানো হয়নি (কোনো Repository-তে Push করা হয়নি); এর দুটি Command স্থানীয়ভাবে চালানো ও যাচাই করা হয়েছে। লক্ষ করুন, CI-র Ubuntu যন্ত্রে সাধারণত Chrome থাকে না, তাই E2E Test Skip হবে; সেখানে E2E চালাতে Browser Install-এর একটি ধাপ যোগ করতে হয়।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
cd currency-desk
node --test
node --test --experimental-test-coverage
node server.mjs
```

`node --test` ৩৮টি Test চালায়: `money` ৯, `fees` ৯, `rates` ৬, `property` ৩, `quote` ৩, `integration` ৭ (একটি Parent Test ও তার ছয়টি Subtest), এবং `e2e` ১। শেষে `# pass 38`, `# fail 0`। Coverage-সহ চালালে, এই বইয়ের লেখার সময় সব `src` File-এ লাইন Coverage ১০০%। `node server.mjs` একটি ঠিকানা দেখায়; Browser-এ খুলে Quote দেখুন।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
E2E Test "skipped"	Chrome বা Edge পাওয়া যায়নি	`CHROME_PATH` Environment Variable
Test শেষ হয় না, ঝুলে থাকে	কোনো Server বন্ধ হয়নি	`t.after`-এ `close()`

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`assert.rejects` সত্ত্বেও ব্যর্থতা ধরা পড়ে না	`await` বাদ	`await assert.rejects(...)`
কখনো পাস, কখনো ব্যর্থ	আসল সময় বা ভাগ করা অবস্থা	`mock.timers`, প্রতিটি Test-এ নতুন Object
Coverage কম	কোনো শাখার Test নেই	Report-এর "uncovered lines" দেখুন

২২.১৪ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
শুধু "Happy Path" Test	ত্রুটি থাকে সীমা ও ব্যর্থতায়	Partition, Boundary, Error-এর পথ
Implementation-এর বিস্তারিত Test	Refactor করলেই Test ভাঙে	আচরণ (Input ও Output) Test
সবকিছু Mock	Test শুধু Mock-কে Test করে	শুধু সীমানায় Mock
আসল সময়ে অপেক্ষা (`sleep(500)`)	ধীর ও Flaky	`mock.timers`, শর্তের অপেক্ষা
এক Test-এ অনেক আচরণ	ব্যর্থ হলে কারণ অস্পষ্ট	একটি আচরণ, একটি Test
Test কখনো ব্যর্থ হতে দেখা হয়নি	হয়তো কিছুই যাচাই করে না	TDD-র Red ধাপ
Coverage-কে লক্ষ্য বানানো	অর্থহীন Test	Coverage দিয়ে ফাঁক খোঁজা
বাইরের API-তে Test	Network-নির্ভর, Flaky	স্থানীয় নকল Server
E2E দিয়ে সব যাচাই	ধীর, ভঙ্গুর	নিয়ম Unit-এ, পথ E2E-তে

২২.১৫ Debugging নির্দেশিকা

১. **একটি Test আলাদা চালান:** `node --test --test-name-pattern="Retry"`।

২. **ব্যর্থতার বার্তা পড়ুন:** `assert` `expected` ও `actual` দেখায়; Table-driven Test-এ Test-এর নামই ইনপুট বলে।

৩. **Flaky হলে বারবার চালান:** একটি Loop-এ ২০ বার; ব্যর্থ হলে সময়, ক্রম বা ভাগ করা অবস্থা খুঁজুন।

৪. **ঝুলে থাকা Test:** কোনো Server, Timer বা খোলা Connection বন্ধ হয়নি; `t.after` দেখুন।

৫. **Property ব্যর্থ:** বার্তায় ইনপুট ছাপুন, এবং সেই ইনপুট দিয়ে একটি Example-based Test যোগ করুন।

৬. **E2E ব্যর্থ:** Page-এর অবস্থা `eval` দিয়ে পড়ুন (`document.body.innerHTML`), Server-এর Log দেখুন।

২২.১৬ পেশাদার অভ্যাস

- **প্রতিটি Bug-এর জন্য প্রথমে একটি ব্যর্থ Test,** তারপর সমাধান।
- **Test-এর নাম একটি বাক্য,** যা আচরণ বর্ণনা করে।
- **Test দ্রুত রাখুন:** পুরো Unit Suite কয়েক সেকেন্ডে।
- **প্রতিটি Test স্বাধীন:** নিজের তথ্য, নিজের Server, যেকোনো ক্রমে চলে।
- **নির্ভরতা Inject করুন,** যাতে Test-এ সময়, Network ও এলোমেলোতা নিয়ন্ত্রণে থাকে।
- **CI-তে প্রতিটি Pull Request-এ Test;** লাল থাকলে Merge নয়।
- **Flaky Test সঙ্গে সঙ্গে ঠিক করুন অথবা সরান,** কখনো উপেক্ষা নয়।

২২.১৭ অনুশীলনী

প্রাথমিক স্তর

১. `isValidAge(age)` (১৮ থেকে ৬০, পূর্ণসংখ্যা)-এর জন্য Equivalence Partition ও Boundary Value-গুলোর তালিকা করুন।

২. প্রশ্ন ১-এর জন্য একটি Table-driven Test লিখুন।

৩. নিচের Test-এ কী ভুল? `test('rejects', () => { assert.rejects(load('x')); });`

৪. `mock.fn` দিয়ে একটি `log` Function নকল করুন, এবং যাচাই করুন যে `processOrder` এটিকে ঠিক একবার "done" দিয়ে ডাকে।

৫. Unit, Integration ও E2E—নিচের প্রতিটি কোন স্তরে: (ক) Fee-র সীমা, (খ) API-র 502, (গ) Button চাপলে ফল দেখা।

মধ্যম স্তর

৬. TDD-তে `formatAmount`-এর ঋণাত্মক পরিমাণের Test লিখুন (Red), তারপর Code (Green)।

৭. `mock.timers` দিয়ে একটি Debounce Function Test করুন: ৩০০ ms-এর মধ্যে তিনটি ডাকে Function একবার চলে।

৮. `parseAmount`-এর জন্য একটি Property লিখুন: যেকোনো বৈধ লেখার কমা মুছে ফেললেও একই মান।

৯. Rate Client-এ "৪২৯ Too Many Requests" কি Retry-যোগ্য হওয়া উচিত? একটি Test লিখে বর্তমান আচরণ দেখান।

চ্যালেঞ্জ স্তর

১০. Integration Test-এ প্রমাণ করুন যে একই মুদ্রা-জোড়ার দুটি Quote-এ Rate Server-এ একটিই অনুরোধ যায় (Cache)।

১১. একটি `retry(fn, { retries, sleep })` Helper TDD-তে লিখুন: প্রথমে তিনটি Test (সফল, শেষে সফল, সব ব্যর্থ), তারপর Code।

১২. E2E-তে নতুন একটি পথ যোগ করুন: অজানা মুদ্রা-জোড়ায় (USD থেকে JPY) বার্তা দেখা যায়।

২২.১৮ অনুশীলনীর সমাধান

সমাধান ১ — বৈধ শ্রেণি: ১৮ থেকে ৬০-এর পূর্ণসংখ্যা। অবৈধ শ্রেণি: ১৮-র কম, ৬০-এর বেশি, ভগ্নাংশ, সংখ্যা নয় (লেখা, `null`, `NaN`)। Boundary Value: ১৭, ১৮, ১৯, এবং ৫৯, ৬০, ৬১। প্রতিনিধি: ৩০ (বৈধের মাঝখান), ১৮.৫, `'30'`।

সমাধান ২

age.test.mjs · JavaScript — চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';

const isValidAge = (age) => Number.isInteger(age) && age >= 18 && age <= 60;

const rows = [[17, false], [18, true], [19, true], [30, true],
  [59, true], [60, true], [61, false], [18.5, false], ['30', false]];

for (const [age, expected] of rows) {
  test(`বয়স ${JSON.stringify(age)}: ${expected}`, () => {
    assert.equal(isValidAge(age), expected);
  });
}
```

Output

```
ok 1 - বয়স 17: false
ok 2 - বয়স 18: true
ok 3 - বয়স 19: true
ok 4 - বয়স 30: true
ok 5 - বয়স 59: true
ok 6 - বয়স 60: true
ok 7 - বয়স 61: false
ok 8 - বয়স 18.5: false
ok 9 - বয়স "30": false
# tests 9
# pass 9
# fail 0
```

সমাধান ৩ — `assert.rejects` একটি Promise ফেরত দেয়, কিন্তু `await` বা `return` নেই, এবং Test Function `async` নয়। Test সঙ্গে সঙ্গে শেষ হয় ও পাস করে; `load` যদি Reject না-ও করে, ব্যর্থতা হারিয়ে যায়। সমাধান: `test('rejects', async () => { await assert.rejects(load('x')); });`।

সমাধান ৪

log.test.mjs · JavaScript — চলবে: Node.js

```js
import test, { mock } from 'node:test';
import assert from 'node:assert/strict';

function processOrder(order, log) {
  const total = order.items.reduce((sum, item) => sum + item.price, 0);
  log('done');
  return total;
}

test('processOrder শেষে একবার "done" Log করে', () => {
  const log = mock.fn();
  const order = { items: [{ price: 50 }, { price: 25 }] };
  assert.equal(processOrder(order, log), 75);
  assert.equal(log.mock.callCount(), 1);
  assert.deepEqual(log.mock.calls[0].arguments, ['done']);
});
```

Output

```
ok 1 - processOrder শেষে একবার "done" Log করে
# tests 1
# pass 1
# fail 0
```

সমাধান ৫ — (ক) Unit: বিশুদ্ধ হিসাব, অনেক ইনপুট। (খ) Integration: API ও Provider-এর মধ্যে আসল HTTP। (গ) E2E: Browser, HTML, Script ও Server একসঙ্গে।

সমাধান ৬

format.mjs · JavaScript — চলবে: Node.js

```
export function formatAmount(minor, currency) {
  const sign = minor < 0 ? '-' : '';
  const abs = Math.abs(minor);
  const whole = Math.floor(abs / 100).toLocaleString('en-US');
  return `${sign}${whole}.${String(abs % 100).padStart(2, '0')} ${currency}`;
}
```

negative.test.mjs · JavaScript — চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { formatAmount } from './format.mjs';

test('ধনাত্মক পরিমাণ', () => {
  assert.equal(formatAmount(123456, 'BDT'), '1,234.56 BDT');
});

test('ঋণাত্মক পরিমাণ (ফেরত টাকা)', () => {
  assert.equal(formatAmount(-250, 'USD'), '-2.50 USD');
});
```

Output

```
ok 1 - ধনাত্মক পরিমাণ
ok 2 - ঋণাত্মক পরিমাণ (ফেরত টাকা)
# tests 2
# pass 2
# fail 0
```

পুরোনো Green সংস্করণে দ্বিতীয় Test-এর ফল ছিল `-3.-50 USD` (Red)। চিহ্ন আলাদা করে পরম মানে কাজ করাই সমাধান—ঠিক Project-এর `formatAmount`।

সমাধান ৭

debounce.test.mjs · JavaScript — চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
```

```
function debounce(fn, ms) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), ms);
  };
}

test('৩০০ ms-এর মধ্যে তিন ডাকে একবার, শেষ মান দিয়ে', (t) => {
  t.mock.timers.enable({ apis: ['setTimeout'] });
  const search = t.mock.fn();
  const debounced = debounce(search, 300);
  debounced('চা');
  t.mock.timers.tick(100);
  debounced('চাকা');
  t.mock.timers.tick(299);
  assert.equal(search.mock.callCount(), 0);
  t.mock.timers.tick(1);
  assert.deepEqual(search.mock.calls.map((c) => c.arguments[0]), ['চাকা']);
});
```

Output

```
ok 1 - ৩০০ ms-এর মধ্যে তিন ডাকে একবার, শেষ মান দিয়ে
# tests 1
# pass 1
# fail 0
```

সমাধান ৮

commas.test.mjs · JavaScript — চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { parseAmount, formatAmount }
  from '../../project/currency-desk/src/money.mjs';

test('কমা থাকুক বা না থাকুক, একই মান', () => {
  let seed = 8;
  for (let i = 0; i < 2_000; i += 1) {
    seed = (seed * 1_103_515_245 + 12_345) % 2_147_483_648;
    const minor = 1 + (seed % 50_000_000);
    const withCommas = formatAmount(minor, 'X').replace(' X', '');
    const plain = withCommas.replaceAll(',', '');
    assert.equal(parseAmount(plain), parseAmount(withCommas));
  }
});
```

Output

```
ok 1 - কমা থাকুক বা না থাকুক, একই মান
# tests 1
# pass 1
# fail 0
```

সমাধান ৯

too-many.test.mjs · JavaScript চলবে: Node.js

```
import test, { mock } from 'node:test';
import assert from 'node:assert/strict';
import { createRateClient } from '../../project/currency-desk/src/rates.mjs';

test('বর্তমান আচরণ: 429 Retry হয় না', async () => {
  const fetchFn = mock.fn(async () => new Response('', { status: 429 }));
  const rates = createRateClient({ baseUrl: 'http://x', fetchFn });
  await assert.rejects(rates.getRate('USD', 'BDT'),
    { code: 'UPSTREAM', retryable: false });
  assert.equal(fetchFn.mock.callCount(), 1);
});
```

Output

```
ok 1 - বর্তমান আচরণ: 429 Retry হয় না
# tests 1
# pass 1
# fail 0
```

Test বর্তমান আচরণ নথিভুক্ত করে: শুধু 5xx Retry-যোগ্য। ৪২৯ মানে "ধীরে চলুন"; সঙ্গে সঙ্গে Retry করলে অবস্থা খারাপ হয়, কিন্তু Server-এর `Retry-After` Header মেনে পরে চেষ্টা করা যুক্তিসংগত। আচরণ বদলাতে চাইলে আগে Test বদলান (Red), তারপর Code।

সমাধান ১০

cache.test.mjs · JavaScript চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { startDesk } from '../../project/currency-desk/src/start.mjs';

test('একই জোড়ার দুটি Quote, Provider-এ একটি অনুরোধ', async (t) => {
  const desk = await startDesk();
  t.after(() => desk.close());
  const body = JSON.stringify({ from: 'USD', to: 'BDT', amount: '10' });
  for (let i = 0; i < 2; i += 1) {
    const url = `${desk.url}/api/quotes`;
    const response = await fetch(url, { method: 'POST', body });
    assert.equal(response.status, 201);
    await response.json();
  }
  assert.equal(desk.provider.controls.requests, 1);
});
```

Output

```
ok 1 - একই জোড়ার দুটি Quote, Provider-এ একটি অনুরোধ
# tests 1
# pass 1
# fail 0
```

`controls.requests` নকল Provider-এর নিজস্ব গণনা; আসল Server-এর ভেতরে না ঢুকেই Cache প্রমাণিত।

সমাধান ১১

retry.test.mjs · JavaScript চলবে: Node.js

```
import test, { mock } from 'node:test';
import assert from 'node:assert/strict';

async function retry(fn, { retries = 2, sleep = async () => {} } = {}) {
  for (let attempt = 0; ; attempt += 1) {
    try {
      return await fn(attempt);
    } catch (error) {
      if (attempt >= retries) {
        throw error;
      }
      await sleep(100 * 2 ** attempt);
    }
  }
```

```
}

test('প্রথমবারেই সফল', async () => {
  assert.equal(await retry(async () => 'ok'), 'ok');
});

test('দুবার ব্যর্থ, তৃতীয়বার সফল; অপেক্ষা 100, 200', async () => {
  const sleep = mock.fn(async () => {});
  const fn = async (attempt) => {
    if (attempt < 2) {
      throw new Error('fail');
    }
    return 'ok';
  };
  assert.equal(await retry(fn, { sleep }), 'ok');
  assert.deepEqual(sleep.mock.calls.map((c) => c.arguments[0]), [100, 200]);
});

test('সব ব্যর্থ: শেষ Error', async () => {
  const fn = mock.fn(async (attempt) => {
    throw new Error(`fail ${attempt}`);
  });
  await assert.rejects(retry(fn), /fail 2/);
  assert.equal(fn.mock.callCount(), 3);
});
```

Output

```
ok 1 - প্রথমবারেই সফল
ok 2 - দুবার ব্যর্থ, তৃতীয়বার সফল; অপেক্ষা 100, 200
ok 3 - সব ব্যর্থ: শেষ Error
# tests 3
# pass 3
# fail 0
```

TDD-তে তিনটি Test আগে লেখা হয়েছিল এবং `retry` না থাকায় তিনটিই ব্যর্থ হয়েছিল (Red); তারপর এই Code।

সমাধান ৬২

e2e-pair.test.mjs · JavaScript — চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { startDesk } from '../../project/currency-desk/src/start.mjs';
import { findBrowser, launch }
  from '../../project/currency-desk/tests/support/browser.mjs';

test('অজানা মুদ্রা-জোড়ার বার্তা', { skip: !findBrowser() }, async (t) => {
```

```
  const desk = await startDesk();
  const page = await launch();
  t.after(async () => {
    await page.close();
    await desk.close();
  });
  await page.open(desk.url);
  await page.type('#to', 'JPY');
  await page.type('#amount', '10');
  await page.click('#get-quote');
  const text = await page.eval(`new Promise((resolve) => {
    const tick = () => document.querySelector('#result').dataset.state
      ? resolve(document.querySelector('#result').textContent)
      : setTimeout(tick, 50);
    tick();
  })`);
  assert.equal(text, 'USD-JPY-এর হার নেই');
});
```

Output

```
ok 1 - অজানা মুদ্রা-জোড়ার বার্তা
# tests 1
# pass 1
# fail 0
```

২২.১৯ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. Fee-র সীমা ১,০০০ টাকা হলে Boundary Value Analysis কোন ইনপুট নেয়?

ক) ৫০০, ৫,০০০

খ) ৯৯৯.৯৯, ১,০০০.০০, ১,০০০.০১

গ) শুধু ১,০০০

ঘ) এলোমেলো ১০টি মান

প্রশ্ন ২. Equivalence Partitioning-এর উদ্দেশ্য?

ক) একই আচরণের ইনপুটগুলোকে শ্রেণিতে ভাগ করে প্রতিনিধি Test

খ) সব ইনপুট Test করা

গ) Test দ্রুত চালানো

ঘ) Coverage বাড়ানো

প্রশ্ন ৩. TDD-র সঠিক ক্রম?

ক) Code, Test, Refactor

খ) Refactor, Red, Green

গ) Red, Green, Refactor

ঘ) Green, Red, Refactor

প্রশ্ন ৪. `t.mock.timers.tick(1000)` কী করে?

ক) এক সেকেন্ড আসলেই অপেক্ষা করে

খ) Test-এর Timeout বাড়ায়

গ) আসল ঘড়ি বদলায়

ঘ) নকল ঘড়িতে সময় এগিয়ে নির্ধারিত Timer চালায়

প্রশ্ন ৫. ১০০% Line Coverage মানে কী?

ক) প্রতিটি লাইন অন্তত একটি Test-এ চলেছে

খ) কোনো ত্রুটি নেই

গ) সব ইনপুট Test হয়েছে

ঘ) Test-গুলো সঠিক

প্রশ্ন ৬. Flaky Test-এর একটি সাধারণ কারণ?

ক) Table-driven Test

খ) Port 0

গ) আসল সময়ের ওপর নির্ভরতা বা ভাগ করা অবস্থা

ঘ) `t.after`

প্রশ্ন ৭. Integration Test-এ বাইরের আসল Rate API-র বদলে স্থানীয় নকল Server কেন?

ক) আসল API ধীর নয়

খ) Test নিয়ন্ত্রিত, পুনরুৎপাদনযোগ্য, এবং ব্যর্থতা ইচ্ছামতো তৈরি করা যায়

গ) নকল Server বেশি সঠিক হার দেয়

ঘ) HTTP লাগে না

প্রশ্ন ৮. Property-based Test কী যাচাই করে?

ক) একটি নির্দিষ্ট ইনপুটের নির্দিষ্ট ফল

খ) শুধু Error

গ) UI-এর রং

ঘ) অনেক ইনপুটে একটি সাধারণ নিয়ম

> **প্রশ্ন ৯.** E2E Test-এ নির্দিষ্ট সময় অপেক্ষার (`sleep(2000)`) বদলে কী ভালো?
>
> ক) একটি শর্তের জন্য অপেক্ষা (যেমন Result-এর একটি চিহ্ন)
>
> খ) আরও বেশি সময় অপেক্ষা
>
> গ) Test বাদ দেওয়া
>
> ঘ) Browser দ্রুত করা

> **প্রশ্ন ১০.** কোন জিনিস Mock করা উচিত নয়?
>
> ক) Network
>
> খ) ঘড়ি
>
> গ) আপনার নিজের বিশুদ্ধ হিসাবের Function, যাকে Test করছেন
>
> ঘ) ইমেইল পাঠানো

২২.২০ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–ক · ৩–গ · ৪–ঘ · ৫–ক · ৬–গ · ৭–খ · ৮–ঘ · ৯–ক · ১০–গ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	সীমার ঠিক নিচে, সীমায় ও ঠিক ওপরে; এখানেই "লাফ" ধরা পড়েছিল।
২	ক	কম Test-এ প্রতিটি আচরণ-শ্রেণি যাচাই।
৩	গ	ব্যর্থ Test আগে, তারপর সরল Code, তারপর পরিষ্কার।
৪	ঘ	আসল অপেক্ষা ছাড়াই সময়-নির্ভর আচরণ।
৫	ক	শুধু "চলেছে"; ঠিক ছিল কি না, তা বলে না।
৬	গ	তাই নকল ঘড়ি ও প্রতিটি Test-এ নতুন অবস্থা।
৭	খ	`failNext` দিয়ে 503 তৈরি করা যায়।
৮	ঘ	যেমন "Fee কখনো কমে না"।
৯	ক	দ্রুত Machine-এ অপচয় নেই, ধীর Machine-এ ব্যর্থতা নেই।
১০	গ	তখন Test শুধু Mock-কেই Test করে।

২২.২১ অধ্যায়ের সারসংক্ষেপ

- নিয়ম Unit-এ, সংযোগ ও Error-এর পথ Integration-এ, গুরুত্বপূর্ণ ব্যবহারকারীর পথ E2E-তে।
- Arrange–Act–Assert ও Table-driven Test: প্রতিটি সারি একটি নামযুক্ত Test।

- Equivalence Partitioning কম Test-এ বেশি আচরণ ঢাকে; Boundary Value Analysis সীমার ত্রুটি ও ব্যবসার প্রশ্ন ধরে।
- Property-based Test হাজারো Seed-যুক্ত ইনপুটে নিয়ম যাচাই করে এবং অজানা সীমাও খুঁজে পায়।
- TDD: Red, Green, Refactor; যে Test কখনো ব্যর্থ হয়নি, তার ওপর ভরসা নেই।
- `mock.fn`, `mock.method`, `mock.timers`: Network, Method ও সময় নিয়ন্ত্রণে; Mock শুধু সীমানায়।
- Integration Test: স্থানীয় Server, Port 0, `127.0.0.1`, `t.after`-এ পরিষ্কার।
- Coverage ফাঁক দেখায়, গুণমান নয়।
- E2E: আসল Browser, শর্তের অপেক্ষা, Browser না থাকলে Skip।
- CI প্রতিটি Pull Request-এ Test চালায়; Flaky Test কখনো উপেক্ষা নয়।

পরের অধ্যায়ে Performance-এর উন্নত কাজ: Profiling-এর হাতিয়ার, Browser Rendering, Virtual Scrolling এবং Web Worker।

২২.২২ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Arrange–Act–Assert	প্রস্তুতি, কাজ, প্রত্যাশা: Test-এর তিন ধাপ।
Table-driven Test	ইনপুট ও প্রত্যাশার টেবিল থেকে তৈরি Test।
Equivalence Partition	একই আচরণ পাওয়া ইনপুটের শ্রেণি।
Boundary Value Analysis	শ্রেণির সীমার আশেপাশে Test।
Property-based Test	অনেক ইনপুটে একটি সাধারণ নিয়মের যাচাই।
TDD	Test আগে লিখে Code লেখার চক্র।
Mock Timers	নকল ঘড়িতে Timer চালানো।
Integration Test	একাধিক অংশ একসঙ্গে, আসল সংযোগে।
End-to-end Test	আসল Browser-এ ব্যবহারকারীর পথ।
Flaky Test	কখনো পাস, কখনো ব্যর্থ হওয়া Test।
Coverage	কোন Code কোনো Test-এ চলেছে তার পরিমাপ।
CI	প্রতিটি পরিবর্তনে স্বয়ংক্রিয় Build ও Test।

২২.২৩ আরও পড়ুন

- **Node.js — Test runner:** `https://nodejs.org/api/test.html` — `mock`, `mock.timers`, Coverage, Reporter।
- **Martin Fowler — The Practical Test Pyramid:** `https://martinfowler.com/articles/practical-test-pyramid.html` — স্তরভিত্তিক কৌশল।
- **Playwright — Getting started:** `https://playwright.dev/docs/intro` — পেশাদার E2E Testing।
- **fast-check — Property based testing:** `https://fast-check.dev/` — Shrinking-সহ Property Test।
- **GitHub Docs — Building and testing Node.js:** `https://docs.github.com/en/actions/use-cases-and-examples/building-and-testing/building-and-testing-nodejs` — CI Workflow।